



Design Document (DD)

Progetto di Ingegneria del Software 2, 2019/2020

Version 1.0

Studente	Matricola
Riccardo Colella	944961
Matteo Ferrini	944759
Francesco Gianchino	946512

Design Document (DD)	0
1. Introduction	2
1.1 Purpose - fuffa	2
1.2 Scope - fuffa	2
1.3 Definitions	2
1.3.1 Definitions	2
1.3.2 Acronyms	2
1.3.3 Abbreviations	3
1.4 Revision History	3
1.5 References	3
1.6 Document Structure - fuffa	3
2. Architectural design	4
2.1 Overview	4
2.2 Component view	5
2.3 Deployment view	6
2.4 Runtime view	6
2.5 Component interfaces	6
2.6 Selected architectural styles and patterns	6
2.7 Other design decisions	6
3. User Interface Design - fuffa	8
4. Requirements traceability	9
5. Implementation, integration and test plan	10
6. Effort Spent	11
7. References	12

1. Introduction

1.1 Purpose

This document aims to give more technical details about the design, the implementation and the testing of the application. It is complementary to the *Requirements Analysis Specification Document*. It aims to give an unified vision of the software so that every team member that will work on it can be coordinated.

The document mainly presents:

- An high level architecture of the software
- The main components and their interfaces
- The runtime behaviour
- The design patterns
- How the requirements map to the design elements
- An implementation, integration and tests plan

1.2 Scope

1.3 Definitions

1.3.1 Definitions

- **Applicant:** we define applicant as the entity who makes the requests to extract the data about parking violations
- **Authority:** it is the service's consumer which has to grant the safety of an area. For more details see section 2.3 User characteristics of the RASD document.
- **Municipality:** it is a particular case of Authority. It can access a broader set of functionalities than a normal authority.

- **SafeStreets:** it is the name of the all software described by this paper. It includes different functions.
- **User:** it is the common single consumer of the service. For more details see section 2.3 User characteristics of the RASD document.

1.3.2 Acronyms

- **API:** Application Programming Interface
- **DB:** Database
- **GPS:** Global Positioning System
- **GDPR:** General Data Protection Regulation
- **HW:** Hardware
- **RASD:** Requirement Analysis Specification Document
- **UML:** Unified modelling language

1.3.3 Abbreviations

- **[Gxx]:** goal number xx
- **[Rxx]:** requirement number xx
- **[Dxx]:** domain assumption number xx
- **App:** application
- **Stats:** statistics

1.4 Revision History

- **9/12/19 - Version 1.0:** First release ;

1.5 References

- Specification document “SafeStreets Mandatory Project Assignment”
- IEEE Standard for Information Technology—Systems Design— Software Design

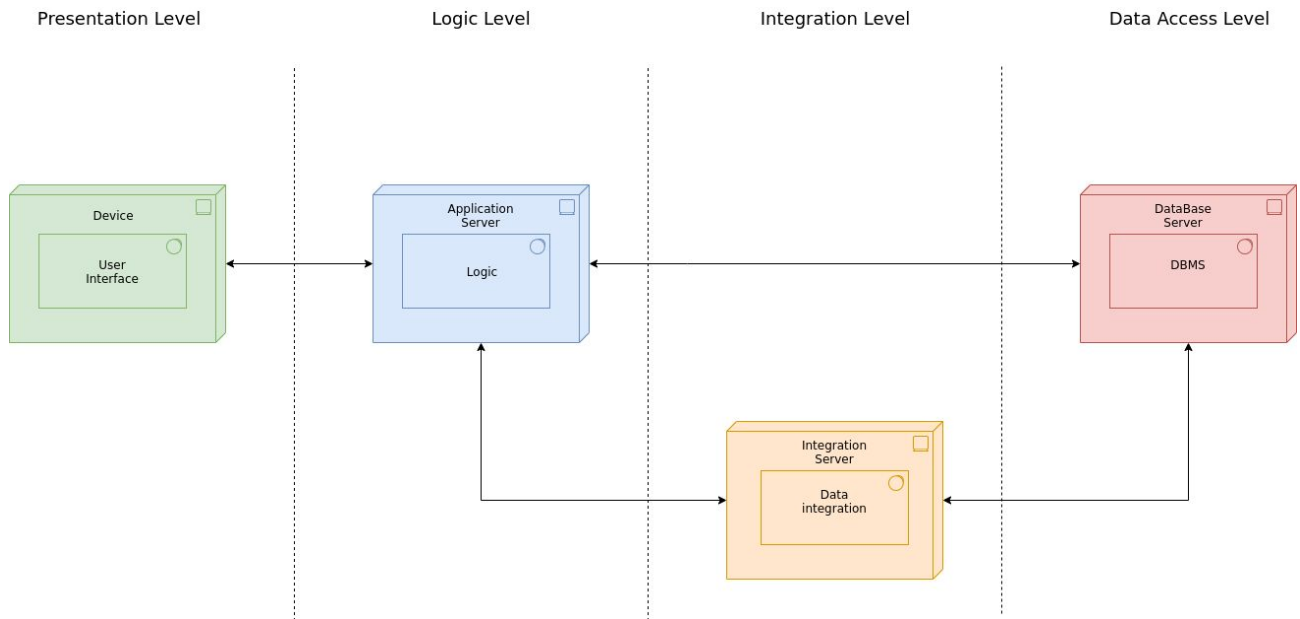
1.6 Document Structure

The document is divided in seven parts:

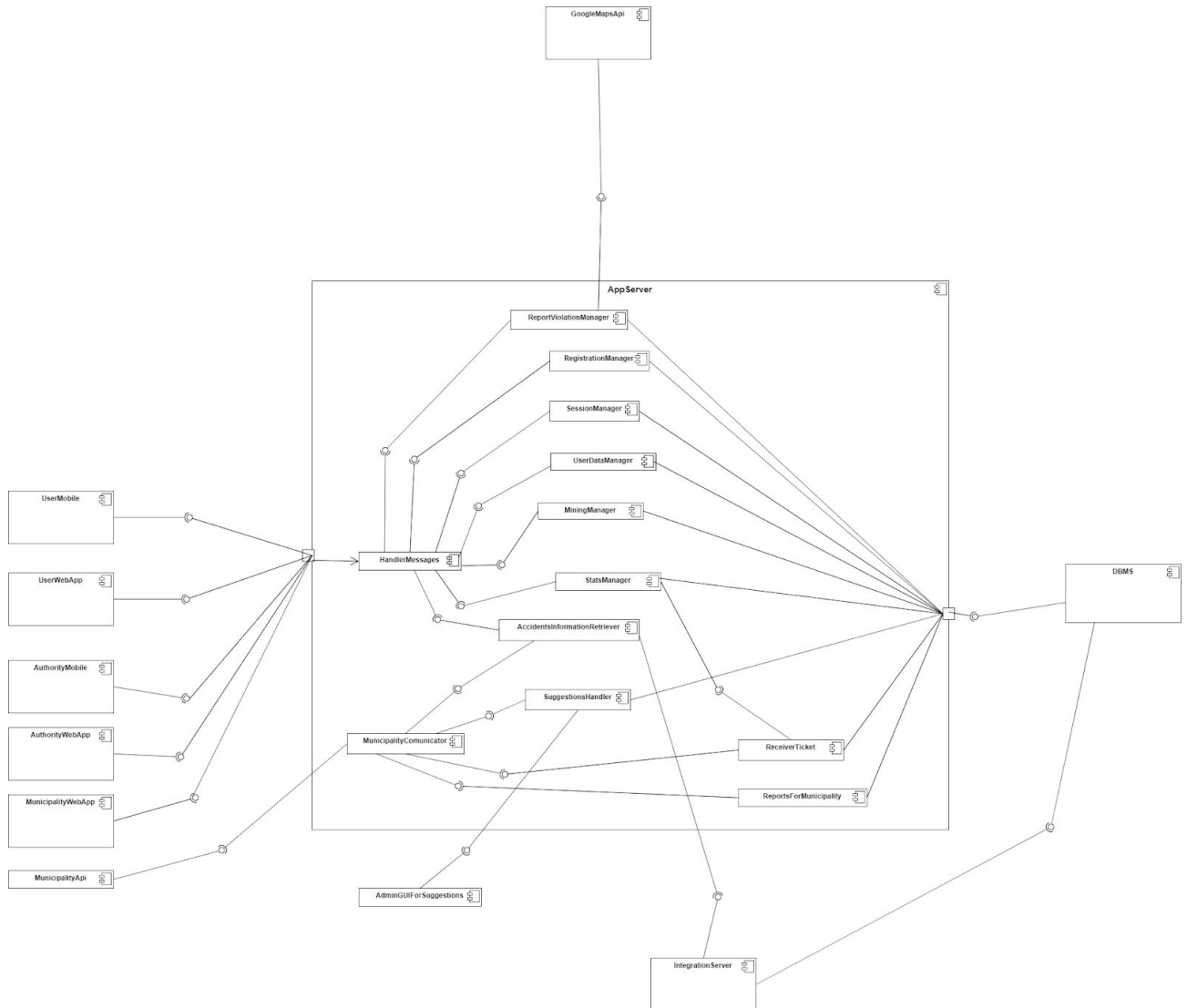
1. **Chapter 1: Introduction.** In this chapter it is given an overall view of the document, its goal and the information needed to understand its contents;
2. **Chapter 2: Architectural Design.** In this chapter it is provided a description of the system's structure, in particular its components, its interfaces, its deployment and the design choices underlying it;
3. **Chapter 3: User Interface Design.** This part will explain how user interface mockups presented in the RASD document were designed;
4. **Chapter 4: Requirement traceability.** This section will explain how the requirements that have been explained in the RASD are satisfied and compliant with the components designed and presented in this document;
5. **Chapter 5: Implementation, integration and tests plan.** In this chapter it is explained the strategy for the implementation, deployment, integration and testing of the various system components.
6. **Chapter 6: Effort Spent.** In this section it is shown the commitment of each group member to the project;
7. **Chapter 7: References.** Here are listed the references taken into account during the project development

2. Architectural design

2.1 Overview



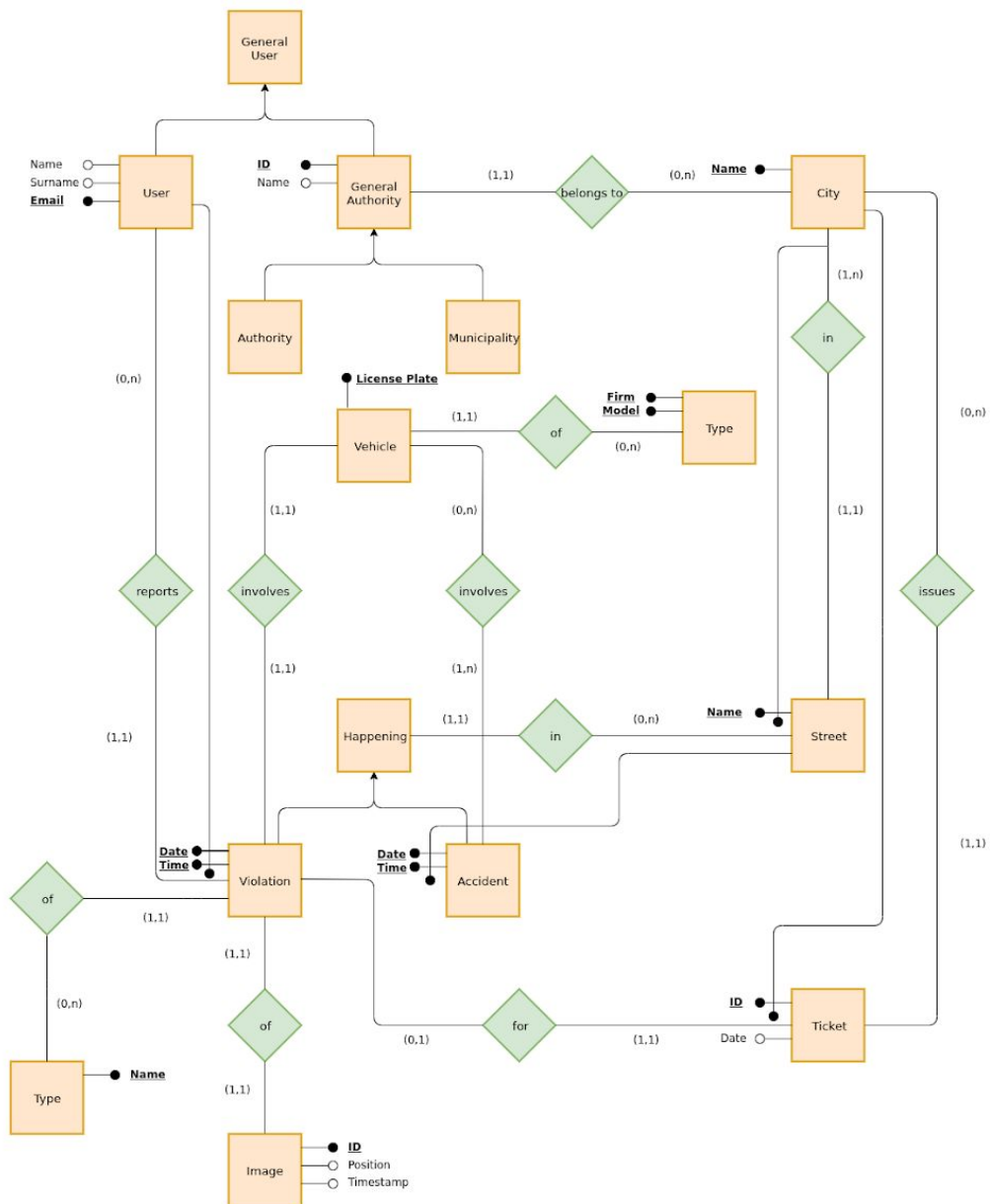
2.2 Component view



The application is made up of the following components:

- HandlerMessages:
- ReportViolationManager:
- RegistrationManager:

- SessionManager:
- UserDataManager:
- MiningManager:
- StatsManager:
- MunicipalityCommunicator:
- AccidentsInformationRetriever:
- SuggestionsHandler:
- ReceiverTicket:
- ReportsForMunicipality:
- AdminGUIForSuggestions:



2.3 Deployment view

2.4 Runtime view

(You can use sequence diagrams to describe the way components interact to accomplish specific tasks typically related to your use cases)

2.5 Component interfaces

2.6 Selected architectural styles and patterns

4 Tier Client-Server

The application will have a multilayered architecture. We distinguish between:

- Presentation Tier: this is the layer responsible for presenting the content retrieved from the application server. It involves either citizens and authorities devices;
- Application Tier: this is the layer responsible for the logic of the system, here information is processed, elaborated and combined;
- Integration Tier: this is the layer needed in order to integrate the data coming from external sources (Municipalities' API and so on) with the internal stored information;
- Data Access Tier: this is the layer in which information is stored.

This solution, which is an extension of the classical 3 tier Client-Server Architecture, allows a better flexibility and an easier implementation, deployment and testing.

Model View Adapter

Given the choice of a relational DB we opted for the Model View Adapter architectural pattern in order to avoid a direct contact between View and Model. The Model is represented by the DataBase, the View by the application devices and the Adapter by the Application and Integration Servers

-mettere immagine

Thin Client

Given the need for image validation, which can't obviously be performed locally, the system will feature a thin client. It will be easier to deploy, lighter and will face less security threats.

Relational DB

The Model will be implemented with relational DB. This choice will ensure a great coherence and low redundancy. Performances will not be an issue given the fact that this particular scenario does not require them.

Loose Coupling

The system will be loosely coupled: this choice will minimize the unnecessary interactions between components and ease the maintenance process. The system will be less performing in data synchronization but the application isn't intended to be real time, so the choice is appropriate.

Stateless components

Given the presence of a relational DB, each component does not need to store an internal state. This justifies the choice of a stateless component pattern.

2.7 Other design decisions

Google APIs

Google is the designated provider for the geolocalisation functionalities. In particular the application will employ the Google Maps API to provide the information needed for the violation report.

Municipality API

The application will be interfaced with Municipalities through their APIs. Given the fact that each API will presumably be different, this is a key aspect which must be managed by a dedicated component, namely the Integration Server.

3. User Interface Design

Since the application mocks have already been inserted in the RASD, the UX diagrams will be integrated in the 2.0 revision of the document above and purposely omitted here for better cohesion.

4. Requirements traceability

In order to demonstrate that the built system is able to fulfill the requirements defined in the RASD here is presented a mapping between those requirements and the system components:

- **R01.** Safestreets take the license plate from the photo sent by user.
 - ReportViolationManager:
- **R02.** The system must allow the user to insert the type of violation.
 - ReportViolationManager:
- **R03.** Safestreet stores the reports provided by users if the photo is not broken.
 - ReportViolationManager:
- **R04.** The system must use a gps system on the device that makes the signal to obtain the position of the violation.
 - ReportViolationManager:
- **R05.** The system must save the collected data of users registered to SafeStreets in real time.
 - RegistrationManager:
 - ReportViolationManager:
- **R06.** The system must allow the user and the authority to access his own registered data and all the reports done with the possibility to consult them.
 - UserDataManager:
- **R07.** The system must allow the applicant to select the interested areas where the parking violations occurred.
 - MiningManager:
- **R08.** The system must show the extracted data to the applicant.

- MiningManager:
- **R09.** If the applicant is a user the system must show only the areas with the highest parking violations frequency.
 - MiningManager:
- **R10.** If the applicant is an authority the system must show the areas with the highest parking violations frequency but also what kind of vehicles committed violations often and the license plates of the most recent infractions.
 - MiningManager:
- **R11.** The system must extract the data about the accidents of a territory from the municipality.
 - AccidentsInformationRetriever:
- **R12.** The system must extract the data about the accidents periodically.
 - AccidentsInformationRetriever:
- **R13.** The system must send the suggestions to the municipality.
 - SuggestionsHandler:
- **R14.** The system must create suggestions for the municipality depending on the information received.
 - SuggestionsHandler:
 - AdminGUIForSuggestions:
- **R15.** The system must send the suggestions periodically.
 - SuggestionsHandler:
- **R16.** The system must create and make available the information about the violation of a territory for the municipality.
 - ReportsForMunicipality:
- **R17.** The system must extract the data about the issued tickets periodically from municipality.

- ReceiverTicket:
- **R18.** The system will use the data about the issued tickets to generate stats about the efficiency of the service.
 - ReceiverTicket:
- **R19.** The system must built stats to show the authority the trends in the issuing of tickets,the most egregious offenders and some parameters about the efficiency of SafeStreets to report the violations.
 - StatsManager:

NOTA: il session manager e gli handler dei messaggi sono presupposti per tutti i requirement e non sono inseriti

5. Implementation, integration and test plan

Identify here the order in which you plan to implement the subcomponents of your system and the order in which you plan to integrate such subcomponents and test the integration.

NOTA: rimarcare il fatto che avendo una struttura a quattro tier il deploy è semplificato (decoupling di app e integration server facilita il testing)

Elencare subcomponents

Per ogni subcomponent elencare i manager

Ipotizzare roadmap implementazione

6. Effort Spent

Matteo Ferrini

Description of the Task	Hours
Introduction	
Architectural Design	
(User Interface Design)	
Requirements Traceability	
Implementation, Integration and Test Plan	

Francesco Gianchino

Description of the Task	Hours
Introduction	
Architectural Design	
(User Interface Design)	
Requirements Traceability	
Implementation, Integration and Test Plan	

Riccardo Colella

Description of the Task	Hours
Introduction	
Architectural Design	
(User Interface Design)	
Requirements Traceability	
Implementation, Integration and Test Plan	

7. References